MS MARCO Generative QA

A Reproducible Retrieve–Rerank–Generate Pipeline with Grounding and Triad Audits

Repository report — updated 2026-06-12

Gioia Zheng

gioia.zheng.stud@gmail.com

Experiment period	2026-03-06 – 2026-05-21 (11 milestones completed; 12-stage plan)
Current update	2026-06-12
Scope	Batch evaluation, CPU-only; English passage retrieval + QA

Project repository

github.com/GioiaZheng/msmarco-genqa

Abstract

This report documents a research-engineering project that reproduces and analyses a complete *retrieve–rerank–generate* question-answering pipeline on the MS MARCO Passage Ranking benchmark [1, 2]. Each stage is implemented from open-source components on a single laptop, with no GPU, no fine-tuning, and no proprietary services: BM25 first-stage retrieval [3, 4] over the 8.8 million-passage corpus; a dense `Sentence-Transformers` baseline with FAISS exact search [5, 6] on a qrels-anchored 50k sample; a cross-encoder reranker [7, 8] over the top-100 candidates; and a frozen T5-small [9] acting as a retrieval-augmented generator [10] on the top-3 retrieved passages.

The single load-bearing empirical result is a paired comparison on all 6 980 *dev/small* queries: replacing BM25 with the cross-encoder-reranked dense top-3 as the upstream passage source — without touching the generator — approximately doubles every surface-form generation metric (Token-F_1 0.197 $\rightarrow$ 0.368, ROUGE-L 0.193 $\rightarrow$ 0.368), with 95 % paired-bootstrap confidence intervals [11] strictly above zero on all four metrics. A DistilBERT-based BERTScore proxy [12, 13] on a 3 000-pair subsample recovers the same paired Δ (+0.173), ruling out a surface-form artefact. A targeted decoding-budget sweep (`max_new_tokens` = 64 $\rightarrow$ 128) and a 40-query regression failure taxonomy together *falsify* the hypothesis that generator truncation drives the residual regression bucket: the generator is hitting end-of-sequence naturally on this prompt format.

A subsequent grounding audit calibrates the headline result. Lexical content-token grounding (0.997) and 3-gram grounding (0.99) sit at the ceiling on both arms, showing that T5-small under the `question: ... context: ...` prompt is performing *extractive* rather than truly generative QA: the rerank gain is almost entirely downstream of retrieval. An NLI-entailment grounding proxy [14, 15] produces the only paired metric in the project whose sign reverses ($\Delta = -0.145$, 95 % CI $[-0.160, -0.130]$), most likely as an artefact of fragmentary midword-cut outputs that defeat sentence-level NLI rather than as a true semantic regression.

The report covers, in order: the dataset characterisation, the four modelling stages, post-hoc evaluation layers (semantic proxy, grounding, and the current triad diagnostic interface), an honest comparison between the initial 12-stage project plan and what actually shipped, and the engineering scaffolding (reproducibility manifests, paired-bootstrap confidence intervals, deterministic seeds) that lets every number in this document be regenerated from one command per stage.

This 2026-06-12 update keeps the empirical claims anchored to the original experiment outputs while bringing the repository description up to date. Since the experiment window closed, the project has gained an installable CLI surface, the `rag-eval` workflow facade, optional model-stack smoke validation, local experiment tracking, a lightweight serving wrapper, refreshed CI, secret scanning, a deterministic RAG triad CLI, stricter input-validation boundaries for run files / JSONL / serving calls, and a clearer advanced-RAG roadmap.

Keywords. MS MARCO; passage retrieval; BM25; dense retrieval; cross-encoder reranking; retrieval-augmented generation; BERTScore; NLI grounding; RAG triad; paired bootstrap; extractive QA.

Contents

List of Tables

List of Figures

1 Introduction

1.1 Background and objectives

The project brief was to build a complete *retrieve–generate* question-answering pipeline on the MS MARCO corpus, exploring how each stage of the pipeline contributes to end-to-end answer quality [1, 10]. MS MARCO is a natural target dataset for this exercise: its queries originate from real Bing user traffic, its relevance judgements are human-annotated, and its v2.1 Q&A flavour ships human-written answers that allow surface-form generation evaluation [2].

The brief named twelve planned milestones spanning EDA, sparse retrieval, generation baselines, dense retrieval, reranking, generator fine-tuning, hallucination mitigation, long-document processing, system integration and acceleration, and a final write-up. In practice the project ran through eleven milestones on a single-user, CPU-only Apple Silicon laptop, with no fine-tuning at any stage. Section 6.3 compares the plan to the actual deliverables and explains the deviations.

1.2 Methodological commitments

Three principles shaped every decision the project made:

1. **Honest baselines before optimisation.** Every stage produces a numerically reproducible *baseline* — BM25 MRR, generation Token-F_1 on a fixed query set, reranking ΔMRR@10 versus a precisely identified first stage — before any tuning is considered. The 200-query subsample baselines are clearly labelled as such; the load-bearing claims are restricted to the full 6 980 *dev/small* numbers.
2. **Paired statistical tests on every comparison.** When the question is "did X help on top of Y?" the report always answers it on *the same query ids*, with a paired bootstrap [11] reporting a 95 % confidence interval on the per-query Δ, not just a point estimate. When the sample is too small for that — e.g. the 200-query smoke-test scale — the report says so and labels the number as illustrative.
3. **Calibrate metrics before celebrating them.** Surface- form metrics (ROUGE-L, BLEU, EM, Token-F_1) under-credit valid paraphrases [16, 17]; a BERTScore semantic proxy [12] and a grounding audit (Section 5.9) check whether the headline gain is real, an artefact of overlap, or a side-effect of the generator's output shape. Two of the three checks confirm the gain; one (the NLI proxy of Section 5.9.2) flips sign, and the report devotes Section 6.2 to that disagreement rather than burying it.

1.3 Contributions

This report contributes the following:

- A single-machine, CPU-only reproduction of the standard MS MARCO Passage retrieval → reranking → RAG-generation pipeline, with the BM25 first-stage MRR@10 within ∼0.014 of the published Anserini/Lucene reference baseline [18] on the same data (Section 4.1).
- A *paired* full-dev comparison of generation quality under BM25 versus cross-encoder-reranked retrieval (Section 5.6), with 95 % paired-bootstrap confidence intervals strictly above zero on all four surface-form metrics and a BERTScore proxy that recovers the same Δ.
- A two-step *falsification* of the most plausible explanation for the residual regression bucket (Section 5.8.3): the decoding-budget closure run and the grounding-ceiling reading together rule out `max_new_tokens` as the bottleneck.
- A grounding audit (Section 5.9) that calibrates the rerank gain as a *retrieval* gain almost entirely downstream of passage selection, and isolates a sign-reversing NLI anomaly (Sec-

tion 5.9.2) most plausibly attributable to fragmentary outputs rather than true semantic drift.

- A reproducibility scaffold (per-run manifests with git commit, dependency hashes, command line, and per-output SHA-256s; deterministic seeds throughout; a single config file driving all four runners) sufficient to re-derive every number in this report from one command per stage on a comparable single-machine setup.
- A maintained repository surface around the experiments: installable console entry points, the `rag-eval` workflow facade, optional model-stack smoke validation, local experiment tracking, a lightweight `mgq-serve` FastAPI wrapper, CI, secret scanning, and separate reproducibility/security notes.

1.4 Roadmap

Section 2 surveys the related work that anchors the design choices. Section 3 characterises the MS MARCO subset actually used. Section 4 steps through the four modelling stages in implementation order. Section 5 is the central empirical chapter, presenting the retrieval numbers, the load-bearing paired generation comparison, the first-stage head-to-head, the evaluation layer, the grounding audit, and the triad diagnostic interface. Section 6 discusses what the combination of evaluation-layer closure, grounding, and triad diagnostics actually means about where the gain comes from, treats the NLI sign flip on its own terms, and compares ship versus plan. Section 7 states the limitations. Section 8 concludes. The question-form breakdown is reported inline with the rest of the evaluation layer (Table 13). Appendices cover the configuration and reproducibility scaffolding (Section A), the full five-way bucket counts (Section B), and a short glossary of cross-references (Section C).

1.5 Repository status as of 2026-06-12

The project has moved from a report-centred experiment bundle to a small research-engineering repository. The headline metrics in this document remain historical experiment results, but the codebase now has a clearer operating surface:

- **Workflow facade.** `rag-eval run -config configs/baseline.yaml` builds or dry-runs the configured retrieval, reranking, generation, context-packing, bootstrap, grounding, and triad-evaluation workflow from one entry point.
- **Package and CLI surface.** The project installs via `pip install -e .` and exposes stage-level `mgq-*` commands plus `mgq-serve` for local serving experiments. The current evaluation surface includes `mgq-retrieval-report matrix` for same-qid BM25 / dense / RRF / reranked comparison tables and `mgq-context-packing-report` for packed-vs-plain generation prompt comparisons, plus `mgq-rag-triad` for context-relevance, groundedness, and answer-relevance diagnostics over existing generation outputs.
- **Validation.** Default CI runs fast pytest coverage, `ruff`, headline-metric metadata checks, and a high-confidence secret scan. Heavy model and data checks stay opt-in through the model-stack smoke and slow/integration test markers.
- **Reproducibility.** The manifest contract now records command lines, dependency-file hashes, config hashes, git state, output hashes, and cache/profile metadata. The documentation separates quick local checks from heavyweight data/model reproduction.
- **Research backlog.** Follow-up issues now separate learned sparse retrieval, stronger rerankers, chunking protocols, query transformation, offline/online indexing, citation-aware generation, and adaptive retrieval sufficiency checks.

2 Related Work

The MS MARCO ecosystem. MS MARCO is among the largest publicly available collections of *real-user* search queries with human-judged relevance and human-written answers [1, 2]. Its Passage Ranking corpus contains 8.8M passages with one or two relevance judgements per dev query; the Q&A v2.1 split layered on top supplies free-form answers that make end-to-end generation tractable to evaluate. The benchmark has been the de-facto target for mid-2010s-to-2020s information retrieval research; published BM25 baselines in the Anserini/Lucene/Pyserini [18] stack report MRR@10 ≈ 0.184 on *dev/small*. This project's BM25 number (MRR@10 $= 0.170$, Section 4.1) is within tokenizer-induced distance of that reference.

Sparse retrieval. BM25 remains the canonical sparse first-stage retriever [3]. This project uses `bm25s` [4], a pure-Python re-implementation of the BM25 scoring formula with eager sparse scoring that achieves Lucene-comparable throughput on commodity CPUs without requiring a JVM; the trade-off is a different default tokenizer, which is the main attributable source of the $\sim$0.014 MRR gap relative to the Lucene-tokenizer reference baseline.

Dense retrieval. Sentence-level dense retrievers [5] encode queries and passages into a shared embedding space. ANCE [19] and subsequent MS MARCO-tuned encoders established that dense retrieval can substantially outperform BM25 on this benchmark. This project uses a *generic* (non-domain-tuned) `all-MiniLM-L6-v2` [8] encoder as the dense baseline, deliberately leaving the MS MARCO-tuned variant (`msmarco-MiniLM-L6-cos-v5`) and a stronger generic encoder (BGE small [20]) for the same-tier encoder comparison. Exact similarity search is via FAISS `IndexFlatIP` [6] on L2-normalised embeddings (cosine similarity); no approximate nearest-neighbour index is used, since the 50k-passage sampled setting fits in memory.

Reranking. Cross-encoders applied to (query, passage) pairs and trained on relevance labels — the architecture introduced by Nogueira and Cho [7] — give the highest known single-stage reranking quality on MS MARCO. The reranker used throughout this project is `cross-encoder/ms-marco-MiniLM-L-6-v2`, a small distilled cross-encoder [8] trained on MS MARCO qrels. Reranking depth is fixed to $K = 100$ throughout the report; the K-sweep ({50, 100, 200}) is queued as the top-k sweep (Section 6.3).

Retrieval-augmented generation. RAG [10] folds retrieved passages into the prompt of an encoder–decoder language model. This project's RAG generator is the smallest T5 [9] checkpoint (`t5-small`), used as a frozen pretrained model with the canonical T5 extractive-QA prompt shape (`question: <q> context: <p1> <p2> <p3>`; see Section 4.2 for the verbatim form). The choice of T5-small is deliberate: it caps the absolute generation quality but lets the entire pipeline run end-to-end on CPU within a few hours per full-dev sweep, which is what makes the paired statistical comparisons of Section 5.6 possible at all.

Generation evaluation. Surface-form metrics — ROUGE-L [16] and BLEU [17] — under-credit valid paraphrases. BERTScore [12] replaces n-gram overlap with contextualised-embedding cosine similarity at the token level. This project uses BERTScore with a DistilBERT [13] backbone as a *semantic proxy* (deliberately not the canonical `roberta-large` citation-grade scorer), reserving the heavier encoder for a follow-up. The grounding audit of Section 5.9 borrows from the faithfulness-evaluation literature on natural language generation [21, 22], specifically the practice of using a small NLI cross-encoder [14, 15] to score *entailment(passages → prediction)*.

Statistical testing. Every paired comparison in this report uses Koehn's paired bootstrap [11] (10 000 resamples, seed 42, percentile interval); structural feature comparisons between regression and non-regression queries use Mann–Whitney U with rank-biserial effect size [23]. This is deliberate: with sample sizes in the thousands but per-query metrics that are highly non-normal (bimodal at 0/1 for exact-match, heavy-tailed for BLEU on short outputs), only resampling-based intervals are credible.

3 Dataset and Exploratory Data Analysis

3.1 Datasets used

The project consumes two distinct slices of MS MARCO:

- **MS MARCO Passage Ranking** (the retrieval corpus): 8.8M English passages, ∼400 bytes median length. All retrieval baselines (BM25, dense, rerank, and the generator's upstream context) index this corpus. Used the official `ir_datasets` loader [24] for both the corpus and the *dev/small* relevance judgements.
- **MS MARCO Q&A v2.1 validation split**: ∼101k queries, each with up to ten candidate passages and one or more human-written reference answers. The EDA uses the first 5 000 rows of this split; the generation evaluation uses the references attached to the 6 980 *dev/small* queries on which BM25 and the reranker both produce a top-3 (Section 5.6).

3.2 Query and passage shape

Figures 1 and 2 report the empirical length distributions on the EDA 5 000-row validation sample. Queries are short (median ≈6 tokens; the long tail beyond ∼15 tokens is small), consistent with web-search behaviour; passages are roughly 10× longer (median ≈50 tokens), short enough that the generator can fit three of them into a T5-small 512-token input window without per-passage truncation in the common case.

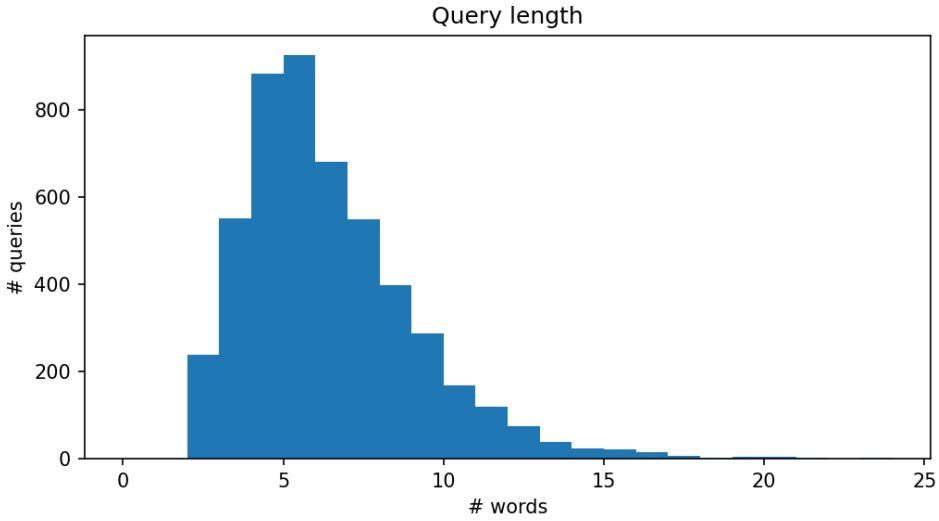

Figure 1. Query-length distribution on the EDA 5 000-row MS MARCO Q&A v2.1 validation sample. Median ≈6 tokens; the ≥15-token tail is small. Sets the sizing target for the RAG prompt budget.

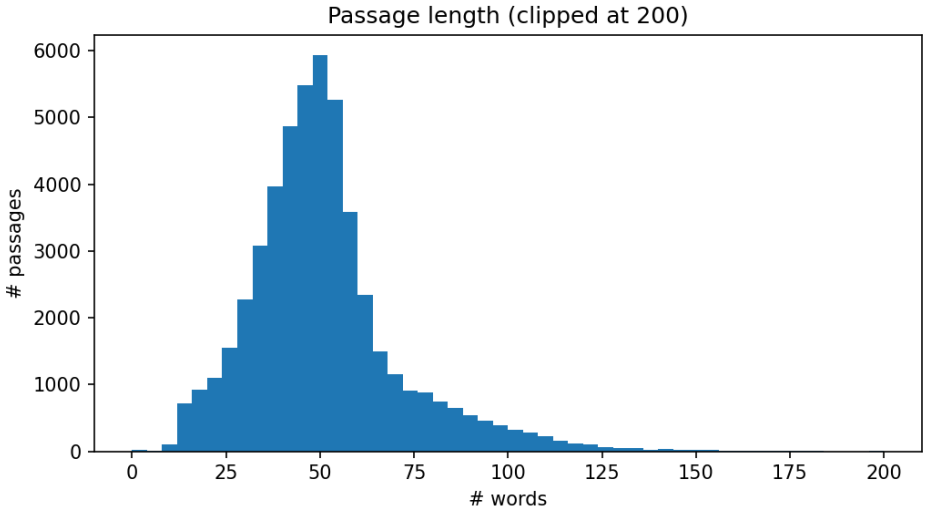

Figure 2. Passage-length distribution on the same EDA sample, clipped at 200 tokens. Median ≈ 50 tokens; the floor at one sentence is an artefact of MS MARCO's snippet extraction.

3.3 Query-type composition

MS MARCO ships a five-way `query_type` label (`DESCRIPTION`, `NUMERIC`, `ENTITY`, `LOCATION`, `PERSON`) and a separate (noisier) free-form answer field. On the 6 980 *dev/small* queries that the load-bearing generation comparison uses, the proportions are `DESCRIPTION` 53 %, `NUMERIC` 24 %, `ENTITY` 9 %, `LOCATION` 7 %, `PERSON` 7 % (see Table 8 in Section 5.6).

Figures 3 and 4 show the EDA distributional read on the validation sample. Two observations end up shaping later stages:

1. `DESCRIPTION` and `NUMERIC` dominate. Description answers are long, free-form, and reward paraphrase — which makes surface-form metrics (ROUGE-L, BLEU, EM) systematically pessimistic on this class and motivates the BERTScore proxy (Section 5.8.1). Numeric answers are short and rely on lexical surface match in the passage, so the bucket analysis of Table 8 can use them as a sanity-check direction (numeric should gain *less* from semantic-aware reranking than description does).
2. A non-trivial fraction of queries has no answer (or an empty / "no_answer" string). These must be filtered for any supervised fine-tuning of the generator; the project does not run SFT, so the no-answer rows are simply scored through (the generator emits something, the reference is empty, and the metric for that row is 0). This is honest but conservative.

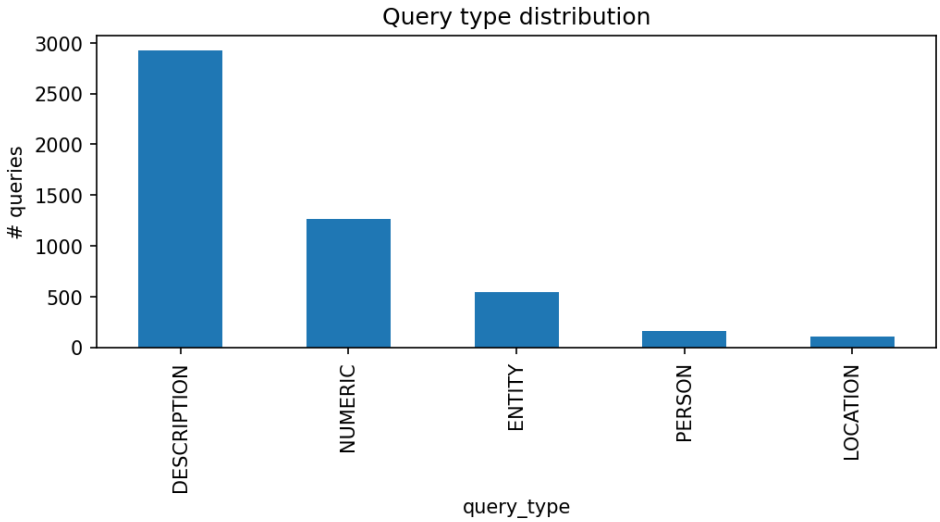

Figure 3. MS MARCO `query_type` label distribution on the EDA validation sample. `DESCRIPTION` dominates; `LOCATION` and `PERSON` are smaller buckets.

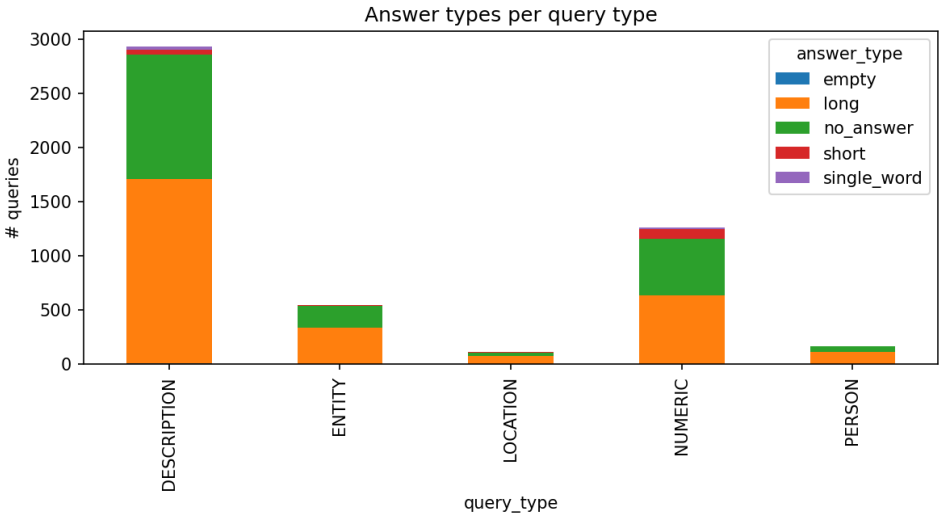

Figure 4. Answer-type composition stratified by `query_type`. `DESCRIPTION` queries skew toward long free-form answers; `NUMERIC` queries skew toward short / single-word.

3.4 Relevance-judgement sparsity

A subtle but consequential property of the *dev/small* qrels: $\geq 95\,\%$ of dev queries have exactly one marked-relevant passage in the 8.8M-passage corpus, and *none* are negatively labelled. This shapes the dense-retrieval sampling design (Section 4.3): a uniform random 50k sample from 8.8M would leave almost no relevant doc in the pool, so the project uses a *qrels-anchored* sample in which every dev relevant doc id is unconditionally included alongside random distractors. The trade-off, spelled out in Section 4.3, is that absolute dense/rerank metrics on this sampled setting are upper-bounded by construction; the *within-sample* Δ between dense and BM25 remains the meaningful quantity.

4 Methodology

This section describes the four modelling stages (sparse retrieval, generation, dense retrieval, and reranking) and the post-hoc analysis layers (evaluation, grounding, triad diagnostics) in implementation order. All four modelling stages ship as scripted command-line entrypoints under `experiments/run_*.py`, driven by a single configuration file at `configs/baseline.yaml`; the evaluation and grounding layers are analysis scripts under `scripts/*.py`, the triad diagnostic is an installable `mgq-rag-triad` diagnostic, and context packing is an installable `mgq-context-packing-report` comparison over packed versus plain generation predictions. These layers write JSON / Markdown summaries beneath the per-analysis directories under `outputs/` (the evaluation, grounding, `rag_triad`, and `context_packing` outputs). Section A documents the reproducibility scaffold.

Figure 5 sketches the end-to-end flow and the place each stage occupies in it.

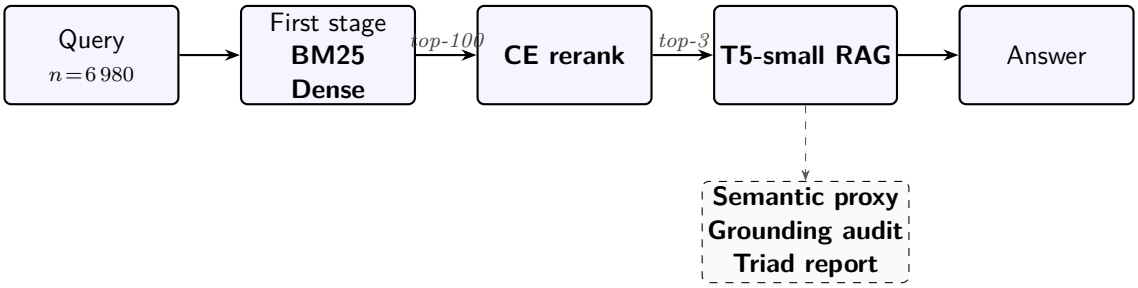

Figure 5. End-to-end pipeline. Solid boxes are the four modelling stages of Section 4; the dashed box is the post-hoc evaluation layer (Sections 5.8 and 5.9), plus the current triad diagnostic interface, which consumes the already-on-disk paired generation predictions and does not re-run any modelling stage. BM25 and dense retrieval are interchangeable first stages feeding into the same downstream pipeline; the paired generation comparison of Section 5.6 switches between them.

4.1 Sparse retrieval (BM25)

Setup. The first stage is BM25 via `bm25s` [4] on the full 8.8M MS MARCO Passage corpus, with the canonical Robertson hyperparameters ($k_1 = 1.5$, $b = 0.75$) and the package-default English tokenizer and stopword list. Index construction and query execution run sequentially on a six-core Apple Silicon CPU; the index is persisted to `data/processed/bm25_index_msmarco/` ($\approx$2.1 GiB) and reused across runs. Top-1000 candidates per query are written to disk in TREC `run.tsv` format.

Retrieve pass. The retrieve pass over 6 980 *dev/small* queries is checkpointed every 200 queries (`retrieval.chunk_size`) into the same `run.tsv` target, so an interrupted run can resume at the next chunk boundary via `-resume` without rebuilding the index or repeating already-completed queries. This is the only operational nicety the project needs at retrieval scale: a single BM25 sweep takes $\sim$70 minutes on the laptop, well below the cost of the downstream reranker, but a kill-9 mid-sweep without checkpointing costs the full hour.

Evaluation. Retrieval metrics are computed in `src/evaluation/retrieval.py` from the run file and the *dev/small* qrels. MRR@10 follows the standard MS MARCO definition (reciprocal rank of the first relevant document in the top-10, zero if none); Recall@100 and Recall@1000 are computed at $K \in \{100, 1000\}$; nDCG@10 uses binary gain following the standard discounted cumulative gain formulation [25].

4.2 RAG generation baseline

Generator. A frozen T5-small [9] checkpoint, used out-of-the-box via Hugging Face Transformers [26]. No fine-tuning at any point. The prompt is the canonical T5-style extractive-QA shape:

```
question: <q> context: <p1> <p2> <p3>
```

where $p_1 \dots p_3$ are the top-3 passages from the upstream retriever, ordered by descending retriever score. Decoding uses the T5 defaults with `max_input_length=512`, `max_new_tokens=64`; the latter is the parameter that the evaluation-layer closure run of Section 5.8.3 sweeps.

Two retrieval sources, same generator. The `run_generation_baseline.py` runner is intentionally *retrieval-source agnostic*: it consumes any TREC-format `run.tsv` and folds the top-K passages into the prompt. The headline generation comparison (Section 5.6) runs the generator twice with exactly the same configuration, switching only the upstream run between (i) the BM25 run and (ii) the cross-encoder-reranked dense run, and uses the runner's `-restrict-to-run` flag to constrain both passes to the *intersection* of query ids covered by both upstream runs — 6 980 queries in total, verified by query-id set-difference equal to $\emptyset$.

Generation metrics. Four surface-form metrics: ROUGE-L via the official `rouge_score` implementation [16], BLEU via the HF `evaluate` wrapper around `sacrebleu` [17], exact-match, and Token-F_1 (lowercased, whitespace-tokenised set F_1). When a query has multiple reference answers, every metric is computed as *best-of-N over references*, which is the convention the MS MARCO Q&A reference scripts use and what makes per-query scores comparable across queries with one versus three references.

4.3 Dense retrieval (sampled)

Encoder & index. The dense retriever uses `sentence-transformers/all-MiniLM-L6-v2` [5, 8] — a generic semantic-similarity encoder, deliberately *not* MS MARCO-tuned — to embed both queries and passages into a 384-dimensional space. Embeddings are L2-normalised and indexed with FAISS `IndexFlatIP` [6], so inner-product search is exact cosine similarity. No approximate nearest-neighbour structure is used.

Why a sampled corpus. Encoding the full 8.8M passage corpus on a six-core CPU laptop is prohibitive (estimated $\sim$40 hours at the dense encode throughput of 16 ms / passage). The dense baseline therefore runs on a *50 000-passage sample*. A uniform random sample at this size would put $\sim 4 \times 10^{-4} \cdot 8.8\text{M} \approx 50$ relevant docs in the pool against $\sim$17 000 dev relevance judgements, so both retrievers would score near zero and the comparison would be meaningless. The project therefore uses a *qrels-anchored sample*: every *dev/small* relevant doc id (7 437 unique docs) is included unconditionally, and the remaining $\sim$42 500 slots are filled with uniform random distractors from the full corpus's doc_id pool (seed 42).

Caveat. This sampling design *inflates* the absolute MRR / nDCG@10 / Recall of both retrievers, because the relevant document is always present in the pool by construction. The legitimate comparison is *dense vs. BM25 on the same sample* (a same-pool head-to-head) — which is exactly what Table 2 reports. Absolute dense numbers are *not* directly comparable to the full-corpus BM25 number.

BM25 on the same sample. To make the head-to- head fair, the runner also rebuilds a BM25 index on the same 50k-passage sample (`data/processed/bm25_sample_index_50k/`) with

identical hyperparameters and re-evaluates it on the same query set. This is the BM25 number in Table 2, not the full-corpus sparse-retrieval baseline.

4.4 Cross-encoder reranking

Reranker. The reranker is `cross-encoder/ms-marco-MiniLM-L-6-v2` [7, 8], a small distilled MS MARCO-trained cross-encoder. It scores (query, passage) pairs jointly — as opposed to the dense bi-encoder which encodes them independently — producing a relevance score for each of the top-K candidates from the first stage.

Depth. Throughout the report, $K = 100$ (`reranker.rerank_top_k`). The choice is empirically motivated and operationally pragmatic: $K = 100$ is the standard MS MARCO reranking depth in the literature and balances rerank quality against wall-clock cost, which is linear in K. The K-sweep ($\{50, 100, 200\}$) is queued as the top-k sweep and explicitly de-scoped per the project's reduced-scope deadline (Section 6.3).

Two arms. Two rerank runs are reported: one over the dense top-100 (Section 5.5, the canonical setup) and one over the BM25 top-100 (Section 5.7, the first-stage head-to-head against the dense arm). Both use the same reranker checkpoint, the same depth, and the same evaluation pass.

Cost. The dense+rerank full-dev run takes ≈4 h 37 min on a 6-core MacBook CPU (538 000 (query, passage) pairs at ∼ 32 pairs/s, peak RSS ∼ 3.3 GiB); the BM25+rerank arm takes ≈3 h 43 min. The cross-encoder is by far the slowest stage in the pipeline.

4.5 Evaluation layer

The evaluation layer adds two post-hoc analyses on the already-on-disk paired generation predictions, neither of which requires re-running the generator or reranker:

Semantic-proxy BERTScore. A DistilBERT-based BERTScore [12, 13] is computed on a 3 000-pair subsample of the shared qid set (seed 42 deterministic), with `rescale_with_baseline=True` and multi-reference max-F_1 per query. This is a deliberate *proxy*: the canonical `roberta-large` BERTScore is reserved for a follow-up (citation-grade, ∼hours of CPU on full dev). The proxy's sole purpose is to answer "does the rerank Δ also show up in a semantic-similarity scorer?"

Regression failure taxonomy. A 40-query seeded triage (seed 42) over the *regression bucket* of Section 5.6 — 233 queries on which the reranker brought the relevant passage into the generator's top-3 *yet* Token-F_1 dropped versus BM25. Labels are produced by a deterministic five-way rule cascade (`scripts/regression_failure_taxonomy.py`); first match wins. Categories: `truncation_short`, `truncation_midword`, `topic_drift`, `extractive_passage_bias`, `semantic_mismatch` — the last is the residual catch-all.

Decoding-budget closure. A targeted full-dev sweep at `max_new_tokens=128` (vs. the canonical 64), holding the generator, prompts, retrieval inputs, and seed fixed. Outputs are written to fresh `_mnt128` directories; the canonical `_full` predictions are untouched. Same paired-bootstrap CI machinery is then applied to the new predictions.

Question-form breakdowns. Three offline analyses on the per-query metric tables: (A) a syntactic question-form tagger over all 6 980 queries (who / what / when / where / why / how / which / yes_no / other), (B) per-form Token-F_1 / ROUGE-L / EM / MRR@10 / nDCG@10 Δ

with paired-bootstrap CIs, and (C) a Mann–Whitney U comparison [23] of five structural features between regression and non-regression queries.

4.6 Grounding audit

The grounding audit is the project's faithfulness layer. The leading question is: *is T5-small extracting content from the retrieved passages, or generating from its parametric memory and happening to overlap?*

Lexical content-token grounding. For each prediction, compute the fraction of unique non-stopword tokens in the prediction (lowercased) that appear anywhere in the union of the prompt's top-3 passages. A local 85-word stopword list keeps the metric self-contained. A score of 1.0 means every distinct content word the model emitted is present in one of the passages it was conditioned on; the metric is bag-of-content-tokens.

3-gram grounding. Tighter than the lexical metric: fraction of the prediction's contiguous 3-grams (case-folded, stopwords retained — order is the signal) that appear as a contiguous span in at least one passage. Catches "the model strung together the right words in the wrong order" cases the bag-of- content-tokens score misses.

NLI-entailment grounding. An NLI cross-encoder (`cross-encoder/nli-deberta-v3-small` [14, 15]) scores *entailment*(passages → prediction) on the same 3 000-paired-qid subsample used by the BERTScore proxy. Same paired-bootstrap-CI machinery. This is the semantic- faithfulness companion to the two lexical grounding metrics.

Edge cases. Predictions with no content tokens score lexical 1.0 vacuously; predictions shorter than $n = 3$ tokens score 3-gram 1.0 vacuously. Both counts are reported per arm so the convention cannot silently inflate the headline number; in practice the vacuous shape is identical on both arms, so the paired Δ is unaffected.

5 Results

This section reports results in the order in which they were produced: the per-stage retrieval baselines (sparse, dense, rerank, Sections 5.1, 5.2 and 5.5), the paired generation comparison that constitutes the principal empirical finding (Section 5.6), the first-stage × reranker head-to-head (Section 5.7), and the two post-hoc calibration layers — semantic-proxy evaluation (Section 5.8) and grounding audit (Section 5.9). Section 6 integrates the findings.

5.1 BM25 baseline (full corpus)

Table 1 reports the BM25 numbers on the full 8.8M-passage corpus and 6 980 *dev/small* queries, against the published Anserini/Lucene reference.

Table 1. BM25 on the full MS MARCO Passage corpus, 6 980 *dev/small* queries. Reference: published Anserini/Lucene MRR@10 ≈ 0.184 [18]. The ∼0.014 gap is attributable to tokenizer differences between Lucene's `EnglishAnalyzer` and the `bm25s` package default.

Metric	Value
MRR@10	0.1703
nDCG@10	0.2138
Recall@100	0.6212
Recall@1000	0.8154

The Recall@100 = 0.621 number deserves attention because it caps what any reranker on top of BM25 can achieve in MRR@10 / nDCG@10: on ∼38 % of *dev/small* queries, the relevant passage is not in the BM25 top-100 *at all*. This recall ceiling is what makes the constrained-recovery comparison in Section 5.7 non-trivial.

5.2 Dense vs. BM25 on the same sample

Table 2 reports the dense baseline against BM25 rebuilt on the *same* qrels-anchored 50k sample. Both columns evaluate on the same 6 980 queries.

Table 2. Dense vs. BM25 on the same qrels-anchored 50k passage sample, 6 980 *dev/small* queries. Encoder: `sentence-transformers/all-MiniLM-L6-v2` (generic, not MS MARCO-tuned). Index: FAISS `IndexFlatIP` over L2-normalised embeddings. Absolute numbers are upper-bounded by the qrels-anchored sampling design (Section 4.3); the meaningful comparison is the same-sample Δ.

Metric	BM25 (sample)	Dense (sample)	Δ (dense − BM25)
MRR@10	0.6948	0.8830	+0.1882
nDCG@10	0.7270	0.9041	+0.1771
Recall@100	0.9338	0.9946	+0.0608
Recall@1000	0.9686	0.9991	+0.0305

Two observations:

1. Dense wins on every metric on the same pool. The MRR@10 Δ = +0.19 is the largest single-step gain in the pipeline; consistent with the literature that generic dense encoders close most of the semantic-matching gap that BM25 leaves open, even without MS MARCO-specific tuning.

2. Dense Recall@100 = 0.9946 is effectively at the ceiling on this sampled setting. From reranking onward, recall is no longer the bottleneck; the reranker's job is to tighten *local ordering* (rank-1 vs. rank-3 vs. rank-10) given that the relevant passage is already in the top-100.

5.3 Encoder horizontal on the dense sample

Table 3 compares three same-tier general-purpose encoders on the identical dense 50k qrels-anchored sample. All three runs share the same sample selection, same BM25-on-sample baseline, and the same evaluation pool, so the comparison is apples-to-apples.

Table 3. Same-tier dense encoders on the dense 50k qrels-anchored sample. Encoding throughput (ms/passage) measured on a single CPU process. Best per column in **bold**.

Encoder	MRR@10	nDCG@10	Recall@100	Recall@1000	ms/passage
`all-MiniLM-L6-v2` (dense baseline)	0.8830	0.9041	0.9946	0.9991	**16.2**
`all-MiniLM-L12-v2`	0.8933	0.9131	0.9955	**0.9996**	31.3
`BAAI/bge-small-en-v1.5`	**0.9021**	**0.9196**	**0.9967**	0.9994	34.4

Read. bge-small-en-v1.5 lifts MRR@10 by +0.019 over the dense baseline at $\sim$2$\times$ encoding cost (CPU, ms/passage). MiniLM-L12 sits in the middle on both axes (+0.010 MRR@10 for the same $\sim$2$\times$ cost as bge-small). The throughput / quality trade is close to linear in this tier; none of the three encoders dominates strictly. The density-sensitivity sweep (Section 5.4) uses bge-small as the encoder.

5.4 Density sensitivity of the dense vs. BM25 gap

The dense 50k baseline is one point on a wider trade-off: *how does the dense-vs-BM25 gap shift as the sample dilutes?* Table 4 sweeps three sample sizes (15k, 30k, 50k) with bge-small (the encoder-comparison winner) and the same BM25-on-sample baseline at each size.

Table 4. Density sensitivity of the same-sample BM25-vs-dense gap. Encoder: `BAAI/bge-small-en-v1.5` (encoder-comparison winner). Density is the share of the sample that is relevant to $\geq$1 *dev/small* query — qrels-anchored sampling unconditionally includes every dev relevant doc, then fills with distractors, so density falls monotonically as the sample grows.

Sample	Density	BM25 MRR@10	Dense MRR@10	Δ (dense $-$ BM25)	Dense Recall@100
15k	49.6 %	0.7806	0.9470	+0.1664	0.998
30k	24.8 %	0.7349	0.9232	+0.1883	0.998
50k	14.9 %	0.6948	0.9021	+**0.2074**	0.997

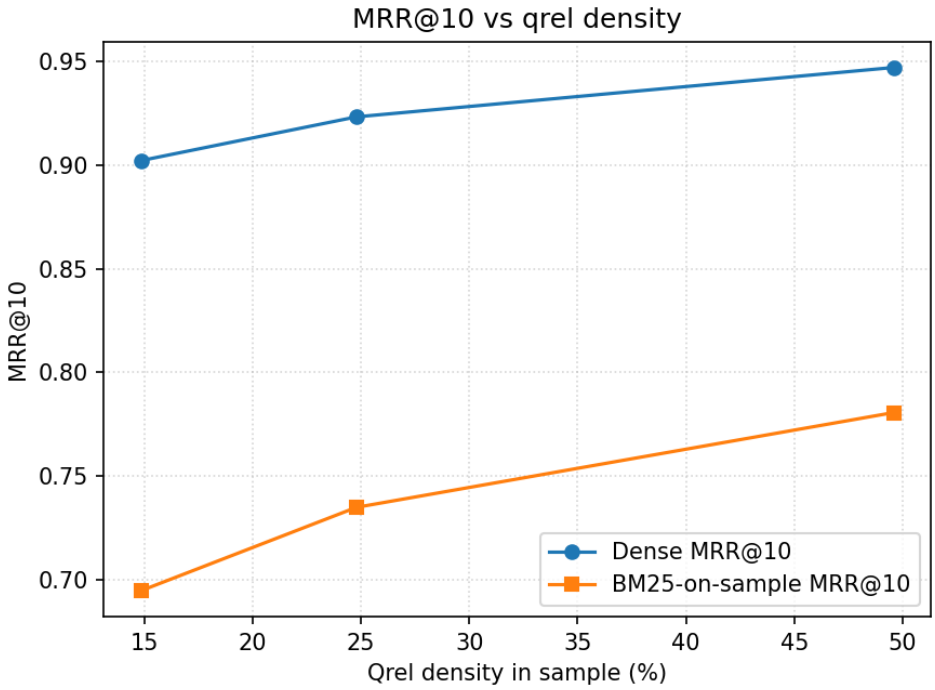

Figure 6. MRR@10 vs. qrel-density on the same three qrels-anchored samples as Table 4. Encoder: `BAAI/bge-small-en-v1.5`. Density decreases left-to-right (50k sample on the left at ∼15 %, 15k sample on the right at ∼50 %). The dense-vs-BM25 gap widens monotonically as density drops — which is the load-bearing read; absolute MRR@10 levels are inflated by the qrels-anchored sampling design (Section 4.3) on every row.

Read. As the sample grows (density drops), dense's advantage over BM25 grows monotonically: +0.166 → +0.188 → +0.207 (Figure 6 shows the two arms diverging visibly). This is consistent with the literature claim that BM25 is competitive at high relevant-density and loses headroom to a generic dense encoder as the distractor mass dilutes the pool. The trend is informative even though the absolute density range (15 %–50 %) is much higher than the 1/5/10 % the project plan originally targeted — those cells need 70k–700k samples to reach (dev/small has ∼7,437 unique relevant doc ids, so a true 1 % density needs a ∼700,000-passage sample) and are deferred.

5.5 Cross-encoder reranking on the dense top-100

Table 5 reports the cross-encoder reranking of the dense top-100, on the full 6 980 *dev/small* queries.

Table 5. Cross-encoder reranking of the dense top-100, 6 980 *dev/small* queries. Reranker: `cross-encoder/ms-marco-MiniLM-L-6-v2` at depth $K = 100$. Recall@100 is unchanged by construction (the reranker is order-only). The earlier 1 000-query subsample produced consistent deltas (MRR $\Delta = +0.0435$, nDCG $\Delta = +0.0398$), so the full-dev gain is not a subsample artefact.

Metric	Dense	Dense + CE rerank	Δ
MRR@10	0.8830	0.9304	**+0.0474**
nDCG@10	0.9041	0.9434	+0.0393
Recall@100	0.9946	0.9946	0.0000

The reranker buys ∼+0.05 MRR even with recall saturated: it is doing exactly what Section 5.2 predicted it should — moving the relevant passage from rank 2–3 to rank 1. Wall-clock cost:

≈ 4 h 37 min on the laptop; peak RSS ≈ 3.3 GiB. The cross-encoder is the slowest stage in the pipeline by approximately two orders of magnitude.

5.6 Headline: Generation × retrieval source

This is the load-bearing result of the project. The question: *does feeding cross-encoder-reranked top-K passages back into the T5-small generator actually improve answer quality?* The comparison is paired by design: same T5-small, same prompt, same top-3 passage count, same seed, same query ids — only the upstream retrieval source changes. Both runs are mutually restricted via `-restrict-to-run` so the two prediction sets cover the *exact same* 6 980 query ids (verified by qid set-difference equal to $\emptyset$).

Surface-form metrics. Table 6 reports the four corpus-level surface-form metrics, paired across the two arms.

Table 6. Headline paired generation comparison on the full 6 980 shared qid set. Same T5-small, same prompt, same top-3 passage count; only the upstream retrieval source differs.

Retrieval → T5-small	ROUGE-L	BLEU	EM	Token-F_1
BM25 top-3	0.1859	0.0717	0.0135	0.1966
Reranked top-3	**0.3621**	**0.2922**	**0.0606**	**0.3677**
Δ (rerank − BM25)	**+0.1763**	**+0.2206**	**+0.0471**	**+0.1711**

Reranking the first stage roughly doubles every surface-form generation metric on full *dev/small*.

Paired-bootstrap confidence intervals. Per-query scores for ROUGE-L and BLEU here come from `rouge_score.RougeScorer` and NLTK `sentence_bleu` (smoothing method 1) so the per-query means differ slightly from the HF corpus-level numbers above; what matters is that all four 95 % CIs lie strictly above zero, with negligible overlap (Table 7).

Table 7. Paired-bootstrap 95 % CI on Δ (rerank − BM25). $n = 6\,980$, 10 000 resamples, seed 42, percentile interval. Two-sided p from the bootstrap is < 0.001 in every case.

Metric	BM25 (per-q mean)	Reranked	Δ	95 % CI on Δ
ROUGE-L	0.1934	0.3677	+0.1742	[+0.1663, +0.1820]
BLEU (sentence)	0.0423	0.1754	+0.1330	[+0.1265, +0.1395]
Exact-Match	0.0135	0.0606	+0.0471	[+0.0417, +0.0527]
Token-F_1	0.1966	0.3677	+0.1711	[+0.1632, +0.1789]

Figure 7 renders the same four CIs as a forest plot.

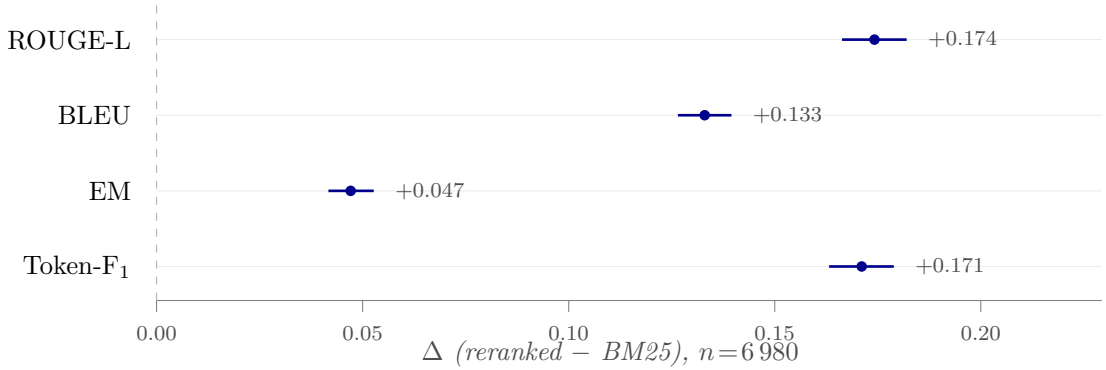

Figure 7. Headline paired-bootstrap forest plot (Table 7). Dots: per-query mean Δ (reranked − BM25); bars: 95 % percentile interval, 10 000 resamples, seed 42. Dashed line at zero: all four intervals lie strictly above it, two-sided bootstrap $p < 0.001$ throughout.

These CIs are ∼6× tighter than an earlier 200-query subsample comparison (n=200 vs. n=6 980), and the full-dev Δ point estimates all sit inside the 200-query CIs — so the original direction-and-magnitude claim survives the move to full *dev/small* intact, with the level numbers naturally deflating to the harder full benchmark.

Retrieval-flag mechanism. The structural mechanism is visible in a single retrieval flag: the rate of having ≥1 relevance-judged passage in the generator's top-3 jumps from **20.8** % (BM25) to **96.9** % (reranked) over the 6 980 paired queries. The reranker's job is precisely to put the right passage in the generator's prompt; once it does, T5-small (which we will see in Section 5.9 is extracting almost all of its content from the prompt) reflects that.

Per-query outcome distribution. Per-query Token-F_1 splits 4 015 strict improvements / 1 766 regressions / 1 199 ties — i.e. a *net* +2 249 / 6 980 queries (≈32 %) strictly improved by feeding reranked passages, with the regressions mostly clustered around already-easy queries where BM25 happened to surface a usable passage. Of those 1 766 strict per-query regressions, **233** fall into the narrower *regression bucket* on which Section 5.8 runs its failure taxonomy: queries where the reranker actually brought the relevant passage into the top-3 *and* the generator's Token-F_1 dropped versus BM25. The distinction matters because the broader 1 766 count contains retrieval-equivalent queries on which the generator simply produced a different surface form; the narrower 233 are the queries that "should have" worked on retrieval grounds but didn't.

Bucket analysis by query type. Table 8 breaks the per-query Token-F_1 Δ down by MS MARCO `query_type`.

Table 8. Paired Token-F_1 broken down by MS MARCO `query_type`. Same 6 980 queries throughout.

`query_type`	n	BM25	Reranked	Δ
DESCRIPTION	3 725	0.1889	0.3939	**+0.2050**
ENTITY	631	0.1765	0.3186	+0.1421
LOCATION	498	0.2495	0.3928	+0.1433
NUMERIC	1 665	0.1997	0.3235	+0.1238
PERSON	461	0.2186	0.3557	+0.1371

`DESCRIPTION` (53 % of the eval set) benefits most; `NUMERIC` benefits least. This direction is consistent with the intuition that short numeric answers depend on lexical surface match with the passage — so semantic-aware reranking of nearly- identical passages buys less than it does for long descriptive answers where the right passage materially changes the available phrasing.

Qualitative example. Query `1043064` (*"what is the chemical formula for oxygen tetrafluoride?"*). BM25 top-3 leads the generator to emit *"Li_2O"*; reranked top-3 leads it to emit *"N_2F_4"* — the correct reference. The retriever, not the generator, changed.

5.7 First-stage × reranker head-to-head

A natural follow-up to Section 5.5: does the same cross-encoder recover *more* from a weaker first stage? Table 9 runs the identical reranker over the BM25 top-100 (full 8.8M corpus) and over the dense top-100 (50k sample), reporting absolute Δ and the *constrained recovery rate* $\Delta\,/\,(\mathit{Recall}@100 - \text{first-stage})$ that controls for each arm's recall ceiling.

Table 9. Same cross-encoder applied to two first stages of very different starting strength. "Constrained recovery" is $\Delta\,/\,(\mathit{Recall}@100 - \text{first-stage})$, which controls for the fact that a reranker can only re-order what the first stage returned into the top-100. Same 6 980 queries on both arms.

First stage	MRR@10 (FS)	MRR@10 (+CE)	Δ_{MRR}	Recov. MRR@10	Recov. nDCG@10
BM25 (8.8M)	0.1703	0.3554	+0.1851	41.1 %	46.1 %
Dense L6 (50k)	0.8830	0.9304	+0.0474	42.4 %	43.4 %

The absolute Δ favours the BM25 arm by a factor of nearly four (more room to recover), but the *constrained recovery rate* comes in essentially tied: ∼42 % for MRR, ∼44 % for nDCG, on both arms. The cleanest reading is that the cross-encoder closes a fixed *fraction* of the ordering gap relative to the recall ceiling of its first stage, regardless of whether that first stage is sparse or dense.

Caveat. The two arms are not on the same corpus: the BM25 arm runs on the full 8.8M-passage corpus, while the dense arm runs on the 50k qrels-anchored sample (Section 4.3). Absolute MRR levels are therefore not comparable across arms, only the within-arm Δ and the recovery rate.

5.8 Evaluation layer

5.8.1 Semantic-proxy BERTScore

The four surface-form metrics in Table 7 are all overlap- or n-gram-based; T5-small's tendency to emit verbose extractive outputs is a known source of mis-credit for surface-form metrics [16, 17]. The load-bearing question of the evaluation layer is therefore: does the rerank Δ also show up in a *semantic* similarity scorer?

Table 10 reports the DistilBERT-based BERTScore proxy on a 3 000-pair subsample of the shared qid set, with the same paired-bootstrap-CI machinery as Table 7.

Table 10. DistilBERT BERTScore proxy [12, 13] on a 3 000-pair subsample of the 6 980 shared qid set. `rescale_with_baseline=True`, multi-reference max-F_1, paired bootstrap 10 000 resamples seed 42, 95 % percentile interval. Per-query: rerank strictly better 64.8 %, tie 5.7 %, BM25 strictly better 29.5 %.

Metric	BM25	Rerank	Δ	95 % CI on Δ	p (two-sided)
BERTScore-F_1 (proxy)	0.2192	0.3920	**+0.1728**	[+0.1608, +0.1850]	< 0.001

The semantic-proxy Δ (+0.1728) lands within ±0.002 of the surface-form Token-F_1 Δ (+0.1711) and the ROUGE-L Δ (+0.1742). **The rerank gain is therefore not a surface-form artefact.** This is the result that justifies acting on the surface-form CIs of Table 7 without first running the canonical `roberta-large` BERTScore. The `roberta-large` pass on full dev is reserved as a paper- writing follow-up (citation-grade scorer, ∼hours of CPU); it will refine the absolute level number but is not expected to change the qualitative direction.

5.8.2 Regression failure taxonomy

Table 11 reports the 40-query seeded triage of the 233-strong regression bucket (Section 5.6).

Table 11. Regression failure taxonomy on a 40-query seeded sample (seed 42) of the 233-strong regression bucket. Labels are a deterministic five-way rule cascade; first-match wins. `semantic_mismatch` is the residual catch-all.

Failure mode	Count	Share
`truncation_midword`	22	55.0 %
`truncation_short` (≤ 3 tok)	14	35.0 %
`topic_drift`	2	5.0 %
`extractive_passage_bias`	2	5.0 %
`semantic_mismatch`	0	0.0 %

90 % of the sample is generator-side truncation, not retrieval or semantic failure. Example (qid 49802, *"belizean cuisine"*): BM25 produced a 24-token descriptive paragraph; the reranked-arm prediction was the two tokens *"Belizean cuisine"* — a passage title. The reranker is doing its job (richer passages); the generator's extractive behaviour turns the richer context into less overlap-matching output on this slice of queries.

At the time of the evaluation-layer report this observation pointed at the `max_new_tokens=64` cap as the cheapest intervention. The closure run of Section 5.8.3 *falsifies* that reading.

5.8.3 Decoding-budget closure: `mnt=64` → `128`

Same generator, same prompts, same retrieval inputs, same seed; the `max_new_tokens` cap is the only thing that changes (64 → 128). Table 12 reports the relevant signals.

Table 12. Decoding-budget closure — same paired generation comparison re-run with `max_new_tokens` = 128. Compared to the canonical `mnt=64` run on the same data. None of the four surface-form deltas move by more than 0.005; the regression bucket shrinks by 2 queries; the 40-query truncation share moves 2.5 p.p.

Signal	`mnt` = 64	`mnt` = 128
Token-F_1 Δ	+0.1711	+0.1706
ROUGE-L Δ	+0.1742	+0.1736
BLEU Δ	+0.1330	+0.1325
EM Δ	+0.0471	+0.0471
Regression bucket size	233	231
Truncation share (40-query triage)	90.0 %	87.5 %

The budget-cap hypothesis is falsified. T5-small is hitting end-of-sequence *naturally* on this prompt format; the mid-word-ending output style observed in Section 5.8.2 is intrinsic to the

model on this prompt shape, not a symptom of the 64-token cap. Generator-side follow-up work therefore has to target either prompt format (multi-passage synthesis, citation-aware decoding) or the model itself (T5-base or larger, instruction-tuned variants), not the `max_new_tokens` knob.

5.8.4 Question-form breakdowns

Three offline analyses on the per-query metrics; no new generation or reranking required.

Question-form tagging. A nine-way syntactic tagger over all 6 980 *dev/small* queries (who / what / when / where / why / how / which / yes_no / other). Complementary to the MS MARCO native `query_type` label of Table 8: the two are correlated but not redundant (e.g. "who" selects exclusively for `PERSON`; "where" selects almost exclusively for `LOCATION`; "what" spans every `query_type`).

Rerank Δ by question-form. Table 13 reports per-form Δ Token-F_1 with paired-bootstrap CIs.

Table 13. Rerank Δ Token-F_1 by question-form on the full 6 980 shared qid set, with paired-bootstrap 95 % CI (10 000 resamples, seed 42).

Form	n	Δ Token-F_1	95 % CI
who	273	+0.1472	[+0.1059, +0.1886]
what	2 751	+0.2097	[+0.1964, +0.2232]
when	189	+0.1207	[+0.0813, +0.1590]
where	283	+0.1774	[+0.1427, +0.2157]
why	75	+0.2267	[+0.1509, +0.3082]
how	837	+0.1520	[+0.1321, +0.1737]
which	120	+0.0250	[−0.0281, +0.0773]
yes_no	480	+0.0680	[+0.0455, +0.0913]
other	1 972	+0.1643	[+0.1497, +0.1801]

`which` (n=120) is the only question form whose 95 % CI on Δ Token-F_1 includes zero ($\Delta = +0.025$, $p = 0.33$), *despite* a retrieval-side Δ MRR@10 of +0.71 on the same subset. Mechanism (hypothesised, not confirmed): `which` queries are typically selection-style ("which of X, Y, or Z is . . . ") and require the generator to choose among items the passage lists; reranking surfaces the right passage but doesn't help the frozen extractive generator make the selection. The remaining factual wh-forms (what / where / why / how / who) all see CIs strictly above zero in the +0.12 to +0.23 range.

Structural profile of regression queries. Mann–Whitney U [23] on five structural features between the 233-query regression bucket and the 6 747 non-regression queries. Four of the five features (query length in tokens, query length in chars, number of qrels per query, BM25 top-3 average passage length) show no detectable difference at any meaningful effect size. The only signal: regression-arm top-3 passage lengths are marginally shorter (Δ median $= -2.7$ tokens, $p = 0.0073$, rank- biserial $r = -0.103$). This is consistent with the Section 5.8.2 truncation finding: shorter passages give T5-small less material to extract. See Figures 8 to 10.

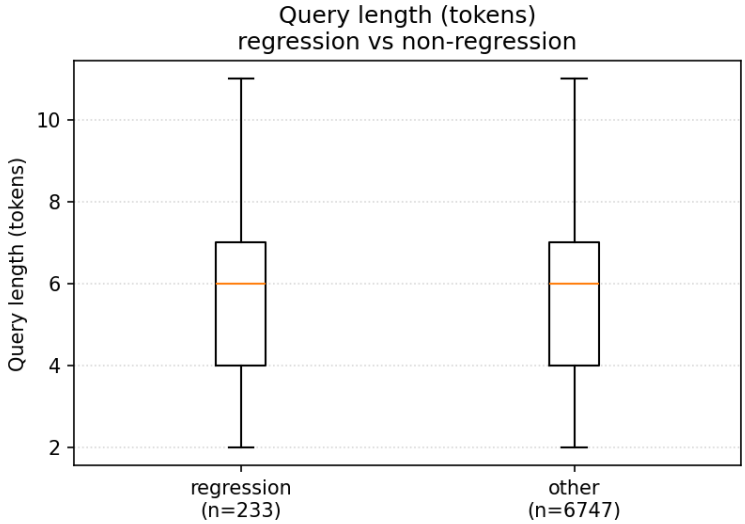

Figure 8. Query-length distribution, regression bucket (n=233) vs. rest (n=6 747). Indistinguishable; Mann–Whitney $p = 0.96$, effect size $r = -0.002$.

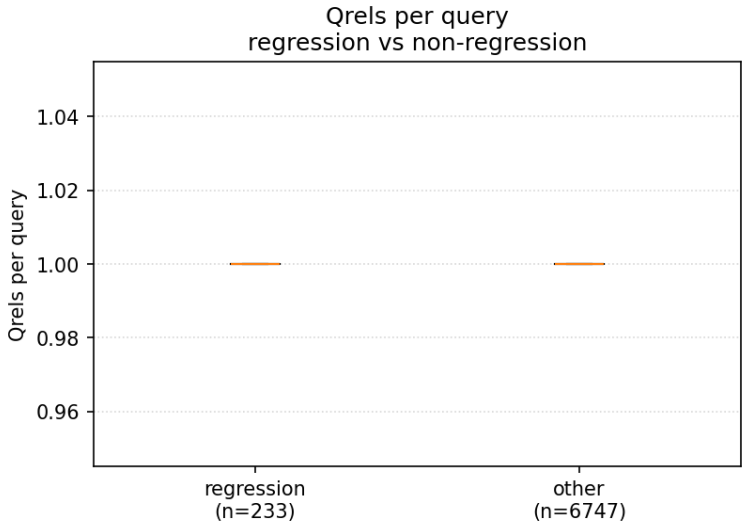

Figure 9. Number of relevance judgements per query. A very small downward shift in the regression bucket ($p = 0.009$, effect size $r = -0.040$); not meaningfully different from zero.

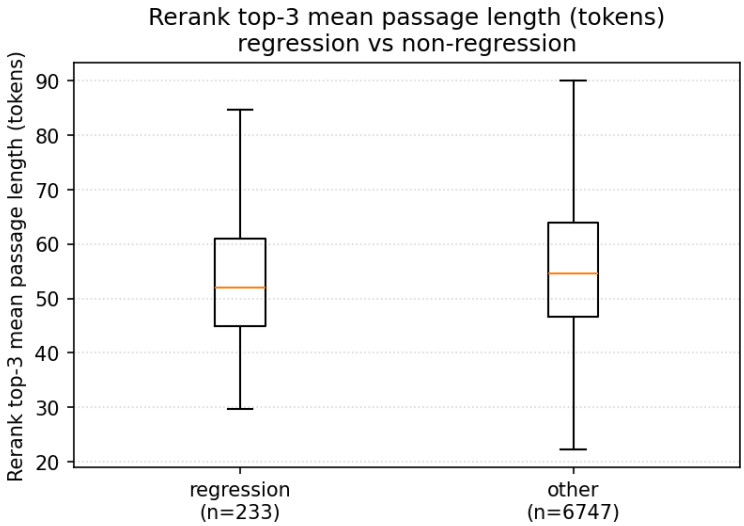

Figure 10. Average rerank top-3 passage length. The only feature with a detectable effect: regression-arm top-3 passages are $\sim$2.7 tokens shorter at the median ($p = 0.0073$, $r = -0.103$). Consistent with Table 11: shorter passages give T5-small less to extract.

5.9 Grounding audit

The evaluation-layer closure (Section 5.8.3) ruled out the decode budget as the bottleneck. The grounding audit answers the question the closure left open: *what is T5-small actually doing on this prompt format — extracting from the passages, or generating from parametric memory?*

5.9.1 Lexical and 3-gram grounding

Table 14 reports both lexical and 3-gram grounding on the full 6 980 paired qid set.

Table 14. Lexical and 3-gram grounding on the full 6 980 paired qid set. Paired bootstrap 10 000 resamples, seed 42, 95 % percentile interval. Both arms sit at the ceiling.

Metric	BM25	Rerank	Δ	95 % CI	p
Lexical content-token	0.9972	0.9977	+0.0005	[−0.0003, +0.0014]	0.24
3-gram	0.9873	0.9905	+0.0032	[+0.0015, +0.0050]	<0.001

Both arms sit at the ceiling. Almost every content word T5-small emits is already in the prompt. 3-gram grounding is slightly higher on the reranked arm (reranking surfaces passages with phrasings closer to the gold answer), but the Δ is +0.003 in absolute terms and both arms remain $\sim$99 %.

Implication. T5-small on the `question: ... context: ...` prompt is performing *extractive* QA, not generative QA. The generation surface-form Δ of Table 6 is therefore almost entirely *downstream of retrieval*: the reranker puts the right words into the prompt and the generator copies them. This is a *calibration* of the headline result rather than a contradiction — the surface-form Δs are real, but the mechanism is "better passages → better extractive output", not "better passages → better neural reasoning".

Edge cases (reported separately). $\sim$ 10 % of predictions per arm are < 3 tokens (the `truncation_short` / `extractive_passage_bias` pattern from Section 5.8.2), scoring 3-gram

grounding as 1.0 vacuously. Lexical-vacuous predictions are negligible ($< 0.3\,\%$). Counts are identical-shape on both arms, so the paired Δ is unaffected.

5.9.2 NLI-entailment grounding

Table 15 reports an NLI-entailment proxy (entailment(passages $\rightarrow$ prediction), `cross-encoder/nli-deberta-v3-small` [14, 15]) on the same 3 000-paired-qid subsample as the BERTScore proxy.

Table 15. NLI-entailment grounding on the same 3 000- paired-qid subsample (seed 42) used by the BERTScore proxy. Same paired-bootstrap CI convention. This is the only paired metric in the entire project whose Δ reverses sign relative to the generation, reranking, and evaluation-layer surface-form story.

Metric	BM25	Rerank	Δ	95 % CI on Δ	p
NLI grounding	0.2270	0.0821	**−0.1448**	[−0.1597, −0.1297]	<0.001

The NLI Δ is negative and strictly excludes zero. Section Section 6.2 discusses the most plausible mechanism and how to disambiguate.

5.9.3 Grounding ↔ downstream correlation

Per-query Spearman / Pearson correlations between each grounding metric and downstream Token-F_1 / BERTScore-F_1, with the high-vs-low (≥ 0.9) bin Mann–Whitney as a second cut.

Direction is uniformly positive but magnitudes are small. Every binned cell has high-grounding − low-grounding > 0; absolute differences sit between 0.04 and 0.10 on the Token-F_1 / BERTScore scale. Per-query Spearman / Pearson correlations are near zero ($|\rho| < 0.05$ in every cell), entirely because both arms sit at the lex / 3-gram ceiling on $\geq 99\,\%$ of queries. The signal is in the *level*, not the per-query rank.

5.9.4 Low-grounding case study

Of the 197 rerank-arm queries with lex < 0.9 OR ngram < 0.9, 30 are sampled (seed 42) and labelled by a coarse five-way rule cascade. Table 16 reports the counts.

Table 16. Low-grounding case study on a 30-query seeded triage (seed 42) of the 197 rerank-arm queries with lex < 0.9 or 3-gram < 0.9. Categories applied in declaration order; first match wins. `parametric_or_external` would flag genuine hallucinations.

Label	Count	Share
`paraphrase_reorder`	23	77 %
`partial_external`	7	23 %
`parametric_or_external`	0	0 %
`prediction_too_short`	0	0 %
`residual`	0	0 %

The 23 `paraphrase_reorder` cases have all the content words in the prompt; only the order or phrasing changes. The 7 `partial_external` cases are almost entirely tokenisation / morphology artefacts: *“350oF”* vs. *“350°F”*, *“competed”* vs. *“compete”*. **Zero genuine hallucinations** (`parametric_or_external`) in the sample. This reinforces the grounding-audit headline: T5-small on this prompt format is doing extractive QA even on its worst-grounded outputs.

5.10 RAG triad diagnostic interface

The current repository adds a deterministic triad report as an engineering-facing diagnostic layer over the same generation prediction schema. The `mgq-rag-triad` CLI writes `metrics.json`, `per_query_triad.jsonl`, `low_score_cases.jsonl`, and `report.md` for one or more generation configurations. The three dimensions are kept separate: context relevance (qrels hit when qrels are supplied, otherwise query–context lexical overlap), groundedness (lexical content-token support in the shown passages), and answer relevance (Token-F_1 against the stored reference answers).

This is intentionally not promoted to a new headline result in this report. Its role is to make future regressions easier to diagnose: a low triad row can be inspected as retrieval-side evidence failure, unsupported generation, answer mismatch, or a combination of those failure modes. The CLI rejects unsupported evaluator names, so future model-assisted RAGAS/TruLens-style scoring must be opted into explicitly rather than silently mixed with deterministic scores.

5.11 Context packing and prompt compression

The current repository also adds a deterministic context-packing layer for generation runs. It is disabled for the historical baselines, but can be enabled with `mgq-generate -context-packing`. The packer uses character budgets as a CPU-friendly proxy for token cost, supports per-passage trimming, query-overlap sentence selection, deduplication, and rank-preserving ordering, and records span metadata that maps the packed prompt context back to source document ids.

The paired report is produced by `mgq-context-packing-report`. It compares an uncompressed `predictions.jsonl` file against a compressed one on matched query ids, then writes `comparison.json`, `per_query.jsonl`, and `report.md`. The report keeps answer-quality deltas (Token-F_1 and exact match) separate from context-character savings, so compression can be evaluated as a cost–quality trade-off rather than as a hidden prompt-format change. Grounding and triad reports should be read alongside this output before treating a packed prompt as a better RAG configuration.

6 Discussion

6.1 Where does the rerank gain come from?

Combining Section 5.6 with Section 5.9, the answer is unambiguous: the headline doubling of generation surface-form metrics is **a retrieval effect, not a generation effect**. T5-small on this prompt format does not invent content; it copies content from the prompt. The reranker's job is to ensure that the content the generator copies is the *right* content. Two pieces of evidence converge:

1. The retrieval-flag rate of having ≥ 1 relevance-judged passage in the top-3 jumps from 20.8 % to 96.9 % across the two arms (Section 5.6). This is the proximate mechanism.
2. Lexical content-token grounding sits at $\sim$0.997 on both arms (Table 14). Whatever content words T5-small emits, $\sim$99.7 % of them are already in the prompt. There is no slack for "the generator thought of something better" to account for any meaningful portion of the generation Δ.

This reading has direct consequences for where further work should go. Decoding-budget interventions are ruled out by Section 5.8.3; generator output-style interventions (Section 5.9.2) are constrained by the ceiling effect; prompt- format and generator-capacity interventions become the natural next axes. Section 6.3 records the generator-capacity (T5-base swap) follow-up that is queued in the project's reduced-scope plan.

6.2 The NLI sign flip

Table 15 is the only paired metric in this project whose Δ reverses sign relative to the otherwise-consistent generation, reranking, and evaluation-layer story: a robust -0.145, with a 95 % CI that strictly excludes zero. Three candidate explanations are worth taking seriously.

(i) Output-shape artefact (currently the best-supported). The 40-query taxonomy (Table 11) and the low-grounding case study (Table 16) jointly show that the reranked-arm predictions contain more fragmentary / mid-word-cut / title-style outputs than the BM25-arm predictions. A sentence-level NLI cross-encoder cannot *entail* a fragmentary hypothesis — it requires syntactically complete propositions on both sides. So even when the rerank-arm prediction is lexically faithful (it copies content tokens from the right passage), the NLI scorer correctly returns low entailment because the prediction *as a sentence* is incomplete. Under this account, the NLI flip is a metric–model mismatch on a particular output style, not a semantic regression.

(ii) True semantic regression on some slice. It is also possible that the reranker, by surfacing tighter / more on- topic passages, paradoxically encourages T5-small to extract title-style or list-style fragments instead of the descriptive paragraphs that would actually be reference-equivalent. Under this account the NLI flip is real: rerank-arm predictions are, on a semantic-faithfulness scale, worse despite being lexically tighter. This is harder to rule out from the existing data because the grounding binned analysis shows a small positive correlation between grounding and downstream metrics (Section 5.9), but the sentence-completeness confound is unresolved.

(iii) NLI checkpoint artefact. The NLI cross-encoder in the NLI-grounding audit is `cross-encoder/nli-deberta-v3-small` — distilled, MNLI-trained [14, 15]. It is not domain-adapted to MS MARCO-style passages and could behave differently from a larger NLI scorer on the project's characteristic short-passage / short-prediction inputs.

How to disambiguate. The cleanest test is a generator swap: rerun the paired generation comparison with T5-base (or an instruction-tuned small model) in place of T5-small, holding everything else fixed, and recompute the NLI proxy. If the NLI Δ moves toward zero or flips back positive without changing the surface-form Δ much, account (i) is supported. If it stays negative at the same magnitude, account (ii) becomes the working hypothesis. This is the planned generator-capacity follow-up, deferred to the post-deadline T5-base run (Section 6.3).

6.3 Plan vs. actual

The project's initial 12-stage plan named the milestones in Table 17. Eleven milestones elapsed; the table records what shipped, what was substituted, and what was de-scoped.

Stage	Plan	Actual	Status
Stage 1	EDA	EDA on the QA v2.1 validation 5k-row sample (Section 3)	shipped
Stage 2	BM25 baseline	Full 8.8M-corpus BM25 retrieve + MRR/Recall (Section 5.1)	shipped
Stage 3	RAG baseline (T5/BART/Phi + top-3)	T5-small RAG generator on the BM25 top-3, 200-query sample baseline + the load-bearing paired full-dev comparison (Section 5.6)	shipped
Stage 4	Dense retrieval, ideally ANCE	Generic `all-MiniLM-L6-v2` on a 50k qrels-anchored sample with FAISS `IndexFlatIP` (Section 5.2); same-tier encoder horizontal added (Section 5.3: MiniLM-L12, bge-small-en-v1.5; bge-small wins +0.019 MRR@10) and density-sensitivity sweep added (Section 5.4: 15k/30k/50k samples). ANCE / MS MARCO-tuned encoders and true low-density cells (1/5/10 %, needing 70k–700k samples) deferred	shipped
Stage 5	Cross-encoder rerank	`ms-marco-MiniLM-L-6-v2` over dense top-100, full-dev (Section 5.5) + BM25-top-100 arm (Section 5.7)	shipped
Stage 6	Generator fine-tuning	*Substituted*: evaluation layer (BERTScore proxy + failure taxonomy + decode-budget closure; Section 5.8). The paired generation comparison made evaluation calibration the higher-leverage axis than SFT, and the evaluation-layer closure + grounding together explain why SFT is unlikely to be the bottleneck.	substituted
Stage 7	Hallucination & citation	*Substituted*: grounding audit (lexical / 3-gram / NLI; Section 5.9). Same goal (faithfulness), more direct measurement. Span-level citation linking is in the explicit out-of-scope list.	substituted
Stage 8	Long-document handling	*De-scoped* due to the project's 2026-05-22 deadline and the choice to prioritise the evaluation and grounding analyses on the already-shipped passage-level pipeline. The infrastructure (sliding-window prompt assembly) exists in the runner; the long-doc benchmark pass does not.	de-scoped

Stage	Plan	Actual	Status
Stage 9	Integration & acceleration (ONNX, quantisation)	Engineering scaffolding shipped (manifests, lockfile, CI, lint; Section A); ONNX / quantisation not pursued on the CPU-only laptop where wall-clock cost is already dominated by the cross-encoder.	partial
Stage 10	Final evaluation	The headline paired comparison serves as the final evaluation; no CodaLab submission.	redirected
Stage 11	Research paper	This report.	shipped
Stage 12	Presentation	Out of report scope.	—

Table 17. Plan vs. actual. "Substituted" indicates that the planned milestone was replaced by an evidence-driven alternative that addressed the same underlying goal; "de-scoped" indicates work that was cut, not redirected.

The two evaluation and grounding substitutions are the project's main methodological deviation from the plan. The reasoning was empirical: once the paired generation comparison shipped, evaluation calibration — "do these deltas hold up under a semantic-similarity scorer?", "where do the remaining regressions come from?" — gave a clearer story than SFT would have, and the evaluation-layer closure run then made that story *actionable* by ruling out the most plausible cheap intervention (the decoding budget). The grounding audit then calibrated the headline result honestly: most of the gain is retrieval, the generator is extractive, and the only sign-reversing metric (Section 5.9.2) is plausibly an output-shape artefact that a generator swap could disambiguate.

7 Limitations

No fine-tuning anywhere. No SFT pass on (question, gold passage, answer) triples has been run. Every model is used as a generic pretrained checkpoint: T5-small for generation, `all-MiniLM-L6-v2` for dense retrieval, `ms-marco-MiniLM-L-6-v2` for the cross-encoder (MS MARCO-trained, but as-is). The absolute level numbers (ROUGE-L $\approx$0.18 on BM25, $\approx$0.36 on reranked) are therefore modest; the load-bearing claim is the paired Δ between rows, which is statistically robust on $n = 6\,980$.

Dense and rerank numbers are on a qrels-anchored sample. The dense 50k sample includes every *dev/small* relevant doc by construction (Section 4.3), inflating absolute MRR / Recall on the sampled setting. The legitimate comparison is within- sample (dense vs. BM25 on the same pool), not against the full-corpus number. This caveat is propagated to the dense+ rerank numbers, which inherit the same sampling.

BM25 tokenizer mismatch with Anserini. The full-corpus MRR@10 = 0.170 versus reference $\approx$ 0.184 is mostly attributable to tokenizer differences between `bm25s` default and Lucene `EnglishAnalyzer`. Acceptable for a single- machine pure-Python pipeline, but it does mean the project's BM25 number cannot be directly compared to published Lucene-tokenizer baselines without bracketing for the tokenizer gap.

Generation metrics are dominated by surface form. The BERTScore proxy (Section 5.8.1) and the grounding metrics (Section 5.9) partially close the surface-form gap, but the canonical `roberta-large` BERTScore on full dev/small is still open.

The NLI sign flip is unresolved. Section Section 6.2 identifies three candidate mechanisms; existing data cannot fully discriminate among them. A T5-base / instruction-tuned-small generator swap is the disambiguating experiment.

***dev/small* only, single seed for the distractor sample.** No held-out test split. The distractor sample is deterministic (seed 42) but not averaged across multiple seeds.

CPU-only. All numbers in this report were produced on a six-core Apple Silicon laptop. No GPU. This is by design — the single-machine constraint is what makes the project's reproducibility claim (one command per stage regenerates every number) tractable — but it caps the kinds of generators that can be evaluated within the deadline.

8 Conclusion

The principal empirical claim of this report is the following. On MS MARCO Passage *dev/small* (6 980 queries), replacing the upstream retriever from BM25 with a cross-encoder- reranked dense top-3 — holding the generator, the prompt, and the random seed fixed — approximately doubles every surface-form generation metric, with 95 % paired-bootstrap confidence intervals strictly above zero on all four metrics (Table 7). The effect is corroborated by a BERTScore semantic-similarity proxy at essentially the same magnitude (Table 10). However, the grounding audit of Section 5.9 shows that both arms operate at the $\sim 99\,\%$ lexical and 3-gram grounding ceiling: under this prompt format T5-small is performing extractive QA, and the observed gain is almost entirely attributable to retrieval rather than to generation.

The methodological contribution of the project lies in the discipline of treating every cross-stage comparison as a paired statistical test and of calibrating each surface-form metric against a semantic-similarity proxy and a grounding metric before accepting it as evidence. Three readings that would have appeared plausible from the generation surface-form numbers alone are rejected on this basis: that the residual regression bucket is driven by the decoding budget (falsified in Section 5.8.3), that reranking materially improves grounding (Table 14 shows both arms at the lexical ceiling), and that the rerank Δ is uniformly positive across metric families (Table 15 shows the NLI proxy reversing sign).

The most informative follow-up experiment is a generator-capacity swap (T5-base or an instruction-tuned small model) on the same prompts and the same retrieval inputs. Such an experiment simultaneously tests whether the surface-form Δ extends with capacity, whether the NLI sign flip persists under a more verbose generator, and whether the lexical-grounding ceiling moves. Each outcome is independently informative for the design of CPU-tractable open-domain QA pipelines on this benchmark.

Future work

The roadmap has become more explicit since the original experiment window. The next steps are grouped by what they test rather than by stage number:

- **Retrieval fusion and retrieval measurement.** Extend the current reciprocal-rank-fusion runner and same-qid matrix report with learned sparse retrieval as a SPLADE-style baseline, then report MRR@10 / nDCG@10 consistently across first-stage, reranked, and fused retrieval outputs.
- **Reranker alternatives.** Evaluate BGE-style cross-encoder reranking and a ColBERT-style late-interaction path against the current MiniLM cross-encoder under matched candidate sets, including latency and memory notes.
- **Chunking and context assembly.** Add semantic chunking and parent-child chunk retrieval experiments, then measure whether better context reconstruction improves the generation and grounding metrics without inflating prompt length.
- **Query transformation and adaptive retrieval.** Add query expansion/de-contextualisation before retrieval and a retrieval-sufficiency check after reranking. The sufficiency layer should trigger a second retrieval pass or abstention rather than forcing unsupported generation.
- **Evaluation hardening.** Extend the current paired bootstrap protocol and deterministic triad report with full-sample `roberta-large` BERTScore, an alternative-family faithfulness scorer, multiple-comparison correction, and TREC-DL qrels loaders.
- **Offline/online separation.** Split batch indexing from online query handling: an offline

job should read data, chunk passages, embed, build FAISS, and persist the index; the online path should load the index and serve retrieval or generation requests through a narrow API.

- **Generator and prompt ablations.** Continue the T5-base / FLAN-T5 capacity sweep, citation-aware prompts, and tokenizer-aware prompt-compression work to test whether the current extractive-grounding ceiling can be moved without sacrificing faithfulness.

The reproducibility scaffold (Section A) is designed so that each follow-up adds a controlled configuration or adapter, not a separate experimental codebase.

Acknowledgements

The project depends entirely on open-source artefacts: MS MARCO itself [1, 2], the Pyserini / Anserini ecosystem and the `bm25s` re-implementation [4, 18], FAISS for exact dense search [6], the Sentence-Transformers and Hugging Face Transformers ecosystems [5, 26], the `ir_datasets` loader [24], the T5 pretrained model [9], the `cross-encoder/ms-marco-MiniLM-L-6-v2` reranker, the DistilBERT and DeBERTa v3 encoders [13, 14], and the `rouge_score` / `sacrebleu` / `evaluate` metric libraries [16, 17].

References

[1] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, M. Rosenberg, X. Song, A. Stoica, S. Tiwary, and T. Wang. "MS MARCO: A Human Generated MAchine Reading COmprehension Dataset". In: *arXiv preprint arXiv:1611.09268* (2016).

[2] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng. "MS MARCO: A Human-Generated MAchine Reading COmprehension Dataset". In: *NeurIPS Workshop on Cognitive Computation: Integrating Neural and Symbolic Approaches*. 2016.

[3] S. Robertson and H. Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.

[4] X. H. Lù. *BM25S: Orders of magnitude faster lexical search via eager sparse scoring*. 2024.

[5] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *EMNLP-IJCNLP*. 2019.

[6] J. Johnson, M. Douze, and H. Jégou. "Billion-scale similarity search with GPUs". In: *IEEE Transactions on Big Data* (2019).

[7] R. Nogueira and K. Cho. "Passage Re-ranking with BERT". In: *arXiv preprint arXiv:1901.04085* (2019).

[8] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers". In: *NeurIPS*. 2020.

[9] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67.

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *NeurIPS*. 2020.

[11] P. Koehn. "Statistical Significance Tests for Machine Translation Evaluation". In: *EMNLP*. 2004.

[12] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi. "BERTScore: Evaluating Text Generation with BERT". In: *ICLR*. 2020.

[13] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter". In: *NeurIPS EMC^2 Workshop*. 2019.

[14] P. He, J. Gao, and W. Chen. "DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-training with Gradient-Disentangled Embedding Sharing". In: *ICLR*. 2023.

[15] A. Williams, N. Nangia, and S. Bowman. "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference". In: *NAACL*. 2018.

[16] C.-Y. Lin. "ROUGE: A Package for Automatic Evaluation of Summaries". In: *Text Summarization Branches Out*. 2004.

[17] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. "BLEU: a Method for Automatic Evaluation of Machine Translation". In: *ACL*. 2002.

[18] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira. "Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations". In: *SIGIR*. 2021.

[19] L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. N. Bennett, J. Ahmed, and A. Overwijk. "Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval". In: *ICLR*. 2021.

[20] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff. *C-Pack: Packaged Resources To Advance General Chinese Embedding*. 2024.

[21] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. "Survey of Hallucination in Natural Language Generation". In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38.

[22] O. Honovich, R. Aharoni, J. Herzig, H. Taitelbaum, D. Kukliansy, V. Cohen, T. Scialom, I. Szpektor, A. Hassidim, and Y. Matias. "TRUE: Re-evaluating Factual Consistency Evaluation". In: *NAACL*. 2022.

[23] H. B. Mann and D. R. Whitney. "On a test of whether one of two random variables is stochastically larger than the other". In: *The Annals of Mathematical Statistics* 18.1 (1947), pp. 50–60.

[24] S. MacAvaney, A. Yates, S. Feldman, D. Downey, A. Cohan, and N. Goharian. "Simplified Data Wrangling with ir_datasets". In: *SIGIR*. 2021.

[25] K. Järvelin and J. Kekäläinen. "Cumulated gain-based evaluation of IR techniques". In: *ACM Transactions on Information Systems (TOIS)* 20.4 (2002), pp. 422–446.

[26] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush. "HuggingFace's Transformers: State-of-the-Art Natural Language Processing". In: *EMNLP: System Demonstrations*. 2020.

A Configuration and reproducibility

A.1 One config, workflow facade, and stage runners

All four modelling-stage runners are driven by a single configuration file at `configs/baseline.yaml`; the evaluation and grounding analysis scripts and the triad CLI read the on-disk paired generation predictions directly. The current repository also exposes a workflow facade through `rag-eval` and installable `mgq-*` console scripts. Table 18 records the canonical command family for each layer.

Table 18. One canonical command per stage. All commands assume the project root as working directory and `pip install -e .` has registered the `src` package.

Stage	Command
Workflow facade	`rag-eval run -config configs/baseline.yaml -dry-run`
Sparse retrieval	`python experiments/run_retrieval.py`
Generation	`python experiments/run_generation_baseline.py`
Dense retrieval	`python experiments/run_dense_retrieval.py`
Reranking	`python experiments/run_reranker.py`
Generation paired	`python scripts/run_full_generation_and_analysis.py`
First-stage head-to-head	`python scripts/compare_rerank_first_stages.py`
Evaluation BERTScore	`python scripts/bertscore_paired_eval.py`
Evaluation taxonomy	`python scripts/regression_failure_taxonomy.py`
Evaluation closure	`python scripts/run_full_generation_and_analysis.py -max-new-tokens 128 -out-suffix _mnt128`
Question-form breakdowns	`scripts/tag_query_forms.py / analyze_rerank_by_query_form.py / regression_query_profile.py`
Grounding	`python scripts/grounding_audit.py`
NLI grounding	`same script with -nli-n-pairs 3000`
Triad	`mgq-rag-triad -predictions bm25=... -predictions reranked=... -output-dir outputs/rag_triad`
Context packing	`mgq-context-packing-report -baseline-predictions ... -compressed-predictions ... -output-dir outputs/context_packing`
Input validation	`docs/input_validation.md`
Model-stack smoke	`python scripts/smoke_model_stack.py -config configs/baseline.yaml -device cpu`
Serving smoke	`mgq-serve` (after installing the `serve` extra)

A.2 Manifests

Every runner writes `outputs/<stage>/manifest.json` alongside `metrics.json`. The manifest records: the current git commit (and a "dirty" flag if any tracked file was modified at launch time), the command line, the configuration file SHA-256, the requirements/lockfile/pyproject SHA-256s, and a per-output SHA-256 (truncated) for every file the runner emits. This is sufficient to verify that a re-run produced byte-identical outputs, or to identify exactly which input changed if the outputs differ.

A.3 Input validation boundary

The repository now distinguishes research-data assumptions from production-facing inputs. TREC `run.tsv` readers reject malformed six-column rows, empty identifiers, invalid or duplicate ranks, duplicate documents within a query, non-finite scores, and non-UTF-8 bytes before expensive

generation or analysis starts. Generation prompt assembly normalises whitespace, rejects empty queries, drops empty optional passages, and applies deterministic character truncation before tokenizer-level truncation. Serving JSONL loaders reject duplicate ids, missing fields, invalid JSON, replacement-character-heavy text, and non-object records. The FastAPI wrapper maps these typed validation failures to structured `422` payloads with `error.type`, `error.message`, and `error.field`.

A.4 Determinism

Seed 42 is used everywhere a random choice enters: the dense qrels-anchored distractor sample, the BERTScore subsample selection, the regression-bucket triage sample, the low-grounding case-study sample, and the 10 000-resample paired bootstrap throughout. This is documented in the per-script CLI help and in the per-stage reports' "Reproduce" sections.

A.5 What is and isn't committed

The project's local operating contract codifies the policy: `outputs/`, `data/`, and the local-scratch directory are gitignored. The tracked artefacts are source code (under `src/`, `experiments/`, `scripts/`, `tests/`), configs, documentation, compact report outputs under `reports/`, and the figures required to render those reports. Large regenerable artefacts (every `metrics.json`, every `run.tsv`, every dataset cache, every model cache) stay outside git.

A.6 Engineering scaffolding

- **Tests.** The default pytest suite is fast and local: no network, no model downloads, and no MS MARCO data fetch. Slow and integration checks are explicitly marked and opt-in.
- **Lint.** `ruff` runs over `src`, `tests`, `experiments`, and `scripts` with a conservative bug-finding rule set.
- **CI.** `.github/workflows/ci.yml` runs default pytest, `ruff`, and headline-metric metadata checks on push and pull request events. The separate `secret-scan.yml` workflow runs the high-confidence secret scanner.
- **Lockfile.** `requirements-lock.txt` is a pip-freeze snapshot of the current security-refreshed CPU dev environment. Historical experiment numbers remain anchored to the first-report tag rather than to moving dependency pins.
- **Optional checks.** The model-stack smoke test loads the pinned generator and dense encoder, runs one short CPU generation, and verifies an embedding shape. It is kept out of default CI because it requires model downloads.
- **Service and tracking.** The `serve` extra exposes `mgq-serve` with `/health` and `/generate`; the `tracking` extra leaves room for MLflow or Weights & Biases while defaulting to local JSONL events.

B Full bucket counts and evaluation-layer analysis tables

Table 19 reports the five-way bucket counts on the full 6 980 paired generation comparison. The classification is from `scripts/analyze_generation_rerank.py`: each query is assigned exactly one bucket based on (i) whether the reranker brought a relevance-judged passage into the generator's top-3 and (ii) whether the generator's Token-F_1 improved, stayed equal, or worsened.

Table 19. Five-way bucket counts on the full 6 980 paired generation comparison. `rerank_fixed_generation_improved` is the "both retrieval and generator improved" headline bucket; `regression` is the narrower 233-strong bucket (Section 5.8.2 runs its taxonomy on this slice).

Bucket	Count
`rerank_fixed_generation_still_failing`	2 684
`rerank_fixed_generation_improved`	2 184
`no_signal`	1 593
`retrieval_equivalent_generation_differs`	286
`regression`	233
Total	6 980

C Glossary of cross-references

- **Sparse retrieval:** BM25 retrieval, full corpus. Output: `outputs/bm25_baseline/`.
- **Generation:** RAG generation baseline (T5-small over top-3). Output: `outputs/generation_{bm25,reranked}_full/`.
- **Dense retrieval:** dense retrieval, 50k qrels-anchored sample. Output: `outputs/dense_retrieval/`.
- **Reranking:** cross-encoder rerank over dense top-100. Output: `outputs/cross_encoder_rerank_full/`.
- **First-stage head-to-head:** same reranker over BM25 top-100. Output: `outputs/rerank_first_stage_compare/`.
- **Evaluation layer:** BERTScore proxy + regression failure taxonomy + decode-budget closure. Outputs: `outputs/bertscore_proxy/`, `outputs/generation_analysis/`, `outputs/failure_taxonomy_mnt128/`.
- **Question-form breakdowns:** question-form tagging, per-form Δ, regression structural profile. Outputs: `outputs/query_type_analysis/`, `outputs/rerank_by_query_form/`, `outputs/regression_features/`.
- **Grounding audit:** lexical + 3-gram grounding audit; the NLI proxy is added on top; grounding $\leftrightarrow$ downstream correlation; low-grounding case study. Outputs: `outputs/grounding/`, `outputs/grounding_correlation/`, `outputs/low_grounding_cases/`.
- **Triad:** deterministic context relevance, groundedness, and answer relevance diagnostics over one or more generation prediction files. Output: `outputs/rag_triad/`.
- **Context packing:** matched-qid comparison of plain versus packed generation prediction files, including answer-quality deltas and context-character savings. Output: `outputs/context_packing/`.